SCALE FOR PROJECT FLY-IN (/PROJECTS/FLY-

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere.42abudhabi.ae:vogsphere/intra-uuid-687c3490-28bd-4338
```

Introduction

- Remain polite, courteous, respectful and constructive
throughout the evaluation process. The well-being of the community
depends on it.

- Identify with the person (or the group) evaluated the eventual
dysfunctions of the work. Take the time to discuss
and debate the problems you have identified.

- You must consider that there might be some difference in how your
peers might have understood the project's instructions and the
scope of its functionalities. Always keep an open mind and grade
him/her as honestly as possible. The pedagogy is valid only and
only if peer-evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student or group's
Git repository.

- Double-check that the Git repository belongs to the student
or the group. Ensure that the work is for the relevant project
and also check that "git clone" is used in an empty folder.

- Check carefully that no malicious aliases was used to fool you
and make you evaluate something other than the content of the
official repository.

- To avoid any surprises, carefully check that both the evaluating
and the evaluated students have reviewed the possible scripts used
to facilitate the grading.

- If the evaluating student has not completed that particular
project yet, it is mandatory for this student to read the
entire subject prior to starting the defence.

- Use the flags available on this scale to signal an empty repository,
non-functioning program, a norm error, cheating etc. In these cases,
the grading is over and the final grade is 0 (or -42 in case of
cheating). However, with the exception of cheating, you are
encouraged to continue to discuss your work (even if you have not
finished it) in order to identify any issues that may have caused
this failure and avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no segfault,
no other unexpected, premature, uncontrolled or unexpected
termination of the program, else the final grade is 0. Use the
appropriate flag.
You should never have to edit any file except the configuration file if it exists.
If you want to edit a file, take the time to explicit the reasons with the
evaluated student and make sure both of you are okay with this.

- This is a Python project: running 'flake8 .' and 'mypy .' must report no error,

the graph/pathfinding logic must not import a forbidden
library (networkx, graphlib, or similar), and the project must be object-oriented. Any of
these failing is a stop condition: tick the appropriate flag and the final grade is 0.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/228892/en.subject.pdf)

maps.tar.gz (https://cdn.intra.42.fr/document/document/57653/maps.tar.gz)

Preliminaries

Basics

The review is done in the presence of the graded learner(s). This
is how everyone progresses: by interacting with others.

- The learner(s) must be present during the defense.
- If the learner is not present, the review cannot proceed.
- As soon as a core functionality is non-functional, the review stops. You can look at the code of the following s
  but they will not be graded.

Yes No

Work submission

Only grade the work that is in the learner's or group's
Git repository. Check that only the requested files are available
in the git repository. If not, the evaluation stops here.

Yes No

README.md

README.md file

Does the repository contain a README.md file at its root, and does it include all of the following?

- The very first line, in italics: "This project has been created as part of the 42 curriculum by <login...>".
- A "Description" section explaining the project's purpose and providing a brief overview.
- An "Instructions" section with details about how to run the project.
- A "Resources" section listing references related to the topic, and stating how AI was used and for which tasks.
- An "Algorithm explanation" section describing the pathfinding approach and design decisions.
- Documentation of visual representation features and how they enhance user experience.
- Example input and expected output demonstrating the program's functionality.

Yes No

Project Structure and Requirements

OOP

The project is completely object-oriented with proper class design
and follows Python OOP best practices. Check for proper class separation,
encapsulation, and inheritance where appropriate.

Yes No

Type safety

The project is completely typesafe: 'mypy .' reports no error.
This is a stop condition (see guidelines above): a failure
here results in a grade of 0.

Yes No

Graph implementation

The graph implementation is custom-built without using forbidden libraries (such as networkx, graphlib, etc.). The learner should be able to explain how it works.

Yes No

Makefile

The repository contains a Makefile at its root with the following rules, each doing what its name says:

- install: installs the project's dependencies.
- run: runs the main program.
- debug: runs the main program in debug mode (e.g. via pdb).
- clean: removes temporary files/caches (e.g. **pycache**, .mypy_cache).
- lint: invokes 'flake8 .' and 'mypy .' with the flags given in the subject.

All five rules must exist and do what their name says.

Yes No

Parser Implementation

Input files

Test the parser with valid input files. The parser must correctly handle ALL of the following, matching the parsed graph to the file exactly, including defaults:

- Number of drones using "nb_drones: <number>" format
- Zone definitions with type prefixes (start_hub:, end_hub:, hub:)
- Connection definitions using "connection: <zone1>-<zone2>" format
- Optional zone metadata with defaults (zone=normal, color=none, max_drones=1)
- Optional connection metadata with max_link_capacity=1 default
- Comments (lines starting with '#') and blank lines are ignored

This is a strict requirement: any element parsed wrong or ignored is a parser failure, it cannot be compensated for by passing the other items.

Yes No

Error handling

Test error handling in the parser. The program must stop and print a clear message naming the problem (and, where possible, the offending line) for ALL of the following, without running the simulation on bad input:

- Malformed files (missing or non-integer drone count, invalid format)
- Invalid zone types (anything other than normal/blocked/restricted/priority)
- Missing start_hub or end_hub definitions, or more than one of either
- Invalid capacity values (zero, negative, or non-integer max_drones/max_link_capacity)
- Duplicate zone names, duplicate connections (a-b and b-a are the same), or a connection referencing an undefi

This is a strict requirement: every case above must be rejected with a clear message. Accepting any of them, or crashing without a message, is a parser failure.

Yes No

Zone and Movements Mechanics

Zone occupancy rules

Test zone occupancy rules with capacity constraints:

- Zones respect max_drones capacity (default 1)
- Drones cannot enter zones that would exceed capacity
- Multiple drones can share start and end zones
- Capacity constraints are properly enforced during simulation

Create test cases to verify these rules are enforced.

Yes No

Movement costs

Test movement costs based on zone types:

- normal zones: 1 turn (default)
- restricted zones: 2 turns
- priority zones: 1 turn (but preferred in pathfinding)
- blocked zones: inaccessible

Verify the simulation respects these movement costs in the output.

Yes No

Connection capacity

Test connection capacity constraints:

- Connections respect max_link_capacity defined in connection metadata (default 1)
- Multiple drones can traverse high-capacity connections simultaneously
- Connection capacity limits are enforced during movement

Verify capacity constraints work correctly in simulation.

Yes No

Visual Representation

Visualization features

Test visual representation features:

- The program provides a clear visual feedback (colored terminal output and/or graphical interface)
- Colors specified in zone metadata are used for visualization
- Visual representation enhances understanding of the simulation
- The visual system clearly shows drone positions and movements

The implementation should demonstrate meaningful visual feedback.

Yes No

Basic Functionality Tests

Simple scenarios

Test with simple scenarios. ALL of the following must pass:
- Test with a single drone on a linear path
- Test with multiple drones using different paths
- Test with the provided example maps (in the subject's attachments)
- Verify output format: each line represents one turn, format "D<ID>-<zone>" or "D<ID>-<connection>"
- Verify stationary drones are omitted from output

Yes No

Simulation ends correctly

Test that the simulation ends correctly when all drones reach the end zone.
The program should stop producing output once all drones have arrived.

Yes No

Pathfinding Algorithm

Valid path

The pathfinding algorithm successfully finds valid paths from start to end zone for various test cases. Test with:

- Simple linear maps
- Maps with multiple possible paths
- Maps with bottlenecks and capacity constraints
- Maps using different zone types

The algorithm should find valid solutions for most test cases.

Yes No

Conflict resolution

Test conflict resolution and capacity management:

- Test scenarios where drones compete for limited capacity
- Verify the algorithm handles multiple drones simultaneously
- Check that capacity constraints prevent conflicts
- Test turn-based movement with multi-turn zones (restricted)

The algorithm should demonstrate proper capacity-aware conflict resolution.

Yes No

Algorithm Explanation

Algorithm explanation

The learner can explain their algorithm complexity, design decisions, and trade-offs made in their implementation. They should be able to discuss their pathfinding approach, capacity management, and optimization strategies.

Yes No

Performance Benchmarks

Maps solved correctly

Independently of turn count, does the program correctly solve all 9 easy/medium/hard maps attached to the subject: every movement/occupancy/capacity rule respected and every drone delivered to the end zone? Speed is irrelevant to this question.

Yes No

Within margin of the optimum

Speed never causes a submission to fail and does not affect the grade. On each of the 9 maps, is the student's turn count within 25% of the map's optimum (<= ceil(1.25 x optimum), see the subject's Performance Benchmarks table)?

Optimum -> max allowed (+25%) per map: linear_path 4 -> 5, simple_fork 4 -> 5, basic_capacity 4 -> 5, dead_end_trap 8 -> 10, circular_loop 10 -> 13, priority_puzzle 6 -> 8, maze_nightmare 13 -> 17, capacity_hell 16 -> 20, ultimate_challenge 26 -> 33.

Answer Yes if the student's result is within its margin on all 9 maps.

Yes No

Exact optimum

Speed never causes a submission to fail and does not affect the grade. Does the student's turn count equal the exact optimum on all 9 provided maps?

Optimum per map: linear_path 4, simple_fork 4, basic_capacity 4, dead_end_trap 8, circular_loop 10, priority_puzzle 6, maze_nightmare 13, capacity_hell 16, ultimate_challenge 26.

Yes No

Edge Cases

Edge cases

Test edge cases. The program must handle ALL of the following correctly (no crash, no incorrect result):

- Single drone scenarios
- Maps with bottlenecks or limited capacity
- Disconnected graphs (no path from start to end): the program must detect this and report it clearly instead of c hanging
- Zones or connections with a very high max_drones/max_link_capacity (effectively unconstrained)

☑ Yes ☒ No

Code Quality and Documentation

Code quality

Code quality assessment:

- Code is well-structured and readable
- Proper use of object-oriented principles
- Appropriate comments and documentation
- Consistent coding style
- Visual representation code is well-integrated

Note: This is not a failing requirement but contributes to overall quality.

☑ Yes ☒ No

Quick Live Coding modifications

Live coding

Please ask the reviewee to add a simple modification to their drone simulation program: display, for each turn, the capacity usage of one zone and one connection (e.g. "Zone X: Y/Z drones, Connection A-B: Y/Z capacity used"), using whichever entry point fits their existing architecture (a CLI flag, a GUI toggle, an extra log line, etc. - do not require a CLI/argparse interface if the submission does not already have one).

The reviewee should be able to locate the relevant parsing and output code, make the necessary modifications, and demonstrate that it works with a test case. The entire task, including the demonstration, should take no more than 10 minutes. Was this procedure followed and did everything work correctly?

☑ Yes ☒ No

Bonus Features

Challenger Map

The Impossible Dream map is solved and reaches its optimum of 43 turns (see the subject's Performance Benchmarks section).

☑ Yes ☒ No

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Norme Cheat Crash Incomplete

▲ Concerning situation ⊘ Forbidden function 💬 Can’t support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33)
Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2)
Privacy policy (https://profile.intra.42.fr/legal/terms/5)
General term of use of the site (https://profile.intra.42.fr/legal/terms/6)
Rules of procedure (https://profile.intra.42.fr/legal/ter